

中国科学院大学

《计算机网络(研讨课)》实验报告

姓名 寇逸欣 学号 2023K8009922004 专业 计算机科学与技术
实验项目编号 11 实验名称 网络地址转换实验

一、实验内容

1. SNAT 实验

- (a) 运行给定网络拓扑 (nat_topo.py)
- (b) 在 n1, h1, h2, h3 上运行相应脚本
 - i. n1: disable_arp.sh, disable_icmp.sh, disable_ip_forward.sh, disable_ipv6.sh
 - ii. h1-h3: disable_offloading.sh, disable_ipv6.sh
- (c) 在 n1 上运行 nat 程序: n1# ./nat exp1.conf
- (d) 在 h3 上运行 HTTP 服务: h3# python3 ./http_server.py
- (e) 在 h1, h2 上分别访问 h3 的 HTTP 服务
 - i. h1# wget http://159.226.39.123:8000
 - ii. h2# wget http://159.226.39.123:8000

2. DNAT 实验

- (a) 运行给定网络拓扑 (nat_topo.py)
- (b) 在 n1, h1, h2, h3 上运行相应脚本
 - i. n1: disable_arp.sh, disable_icmp.sh, disable_ip_forward.sh, disable_ipv6.sh
 - ii. h1-h3: disable_offloading.sh, disable_ipv6.sh
- (c) 在 n1 上运行 nat 程序: n1# ./nat exp2.conf
- (d) 在 h1, h2 上分别运行 HTTP Server: h1/h2# python3 ./http_server.py
- (e) 在 h3 上分别请求 h1, h2 页面
 - i. h3# wget http://159.226.39.43:8000
 - ii. h3# wget http://159.226.39.43:8001

3. 手动构造一个包含两个 nat 的拓扑

- (a) h1 <-> n1 <-> n2 <-> h2
- (b) 节点 n1 作为 SNAT, n2 作为 DNAT, 主机 h2 提供 HTTP 服务, 主机 h1 穿过两个 nat 连接到 h2 并获取相应页面

二、实验步骤

本次实验主要需要补全的代码部分在 nat.c 中, 主要包括以下几个函数:

1. get_packet_direction: 根据判断数据包的方向

2. do_translation: 进行 NAT 转换
3. nat_timeout: 清除超时的 NAT 映射表项
4. parse_config: 解析配置文件
5. nat_exit: 释放 NAT 相关资源

2.1 判断数据包方向

对于传入的数据包, 我们通过解析 IP 头部和路由表匹配来确定数据包是从内部接口 (internal_iface) 发出 (DIR_OUT) 还是从外部接口 (external_iface) 接收 (DIR_IN)。如果无法匹配, 则返回 DIR_INVALID, 表示方向无效。

Listing 1: get_packet_direction 函数代码实现

```
static int get_packet_direction(char *packet)
{
    //fprintf(stdout, "TODO: determine the direction of this packet.\n");
    struct iphdr *ip = packet_to_ip_hdr(packet);
    rt_entry_t *rt = longest_prefix_match(ntohl(ip->saddr));

    if (rt->iface->index == nat.internal_iface->index) {
        return DIR_OUT;
    } else if (rt->iface->index == nat.external_iface->index) {
        return DIR_IN;
    }

    return DIR_INVALID;
}
```

2.2 进行 NAT 转换

在进行 NAT 转换的过程中, 首先需要提取 IP 头部和 TCP 头部, 以便获取数据包的关键信息。函数通过调用 packet_to_ip_hdr(packet) 和 packet_to_tcp_hdr(packet) 来解析这些头部。随后, 根据数据包的方向 (DIR_IN 或 DIR_OUT) 获取相应的端口号和 IP 地址。例如, 对于入站数据包, 使用源 IP 地址和源端口; 对于出站数据包, 使用目的 IP 地址和目的端口。具体代码如下:

```
struct iphdr *ip = packet_to_ip_hdr(packet);
struct tcphdr *tcp = packet_to_tcp_hdr(packet);

// 获取当前数据包信息 (主机序)
u32 saddr = ntohl(ip->saddr);
u32 daddr = ntohl(ip->daddr);
u16 sport = ntohs(tcp->sport);
u16 dport = ntohs(tcp->dport);
```

由于 NAT 可能需要同时维护数万条映射关系, 链表查找方式效率很低, 因此考虑使用哈希方式。在原代码中也提供了一种哈希函数, 使用该哈希函数对远程 IP 进行映射。代码中通过 hash8((char *)&remote_ip, 4) 计算哈希值, 然后在对应的哈希桶中查找映射。

```
// 1. 确定 Remote IP/Port 并计算哈希
u32 remote_ip = (dir == DIR_OUT) ? daddr : saddr;
u16 remote_port = (dir == DIR_OUT) ? dport : sport;

u8 hash = hash8((char *)&remote_ip, 4);
struct list_head *head = &nat.nat_mapping_list[hash];
```

而后,如果方向是 DIR_IN 的话,对 NAT 映射表进行遍历,如果存在外部 IP 地址和端口符合该数据包目的地址 IP 地址和端口的,则设置 mapping 不为 NULL。如果 mapping 不为 NULL,则表示可以直接利用该映射,修改 TCP 头部的目的端口及 IP 头部的目的 IP 地址,并更新映射表中的序列号、连接信息的 ACK 和 FIN 标志,最后更新时间。

如果 mapping 为 NULL,则遍历 DNAT 规则表,检查规则对应的外部端口是否被分配,规则的外部 IP 地址是否与 IP 头的目标地址匹配且规则的外部端口是否与 TCP 头的目的端口匹配。若满足条件,则按照规则创建新的映射项,并将映射项最终添加到映射表的链表中。最后与 mapping 不为 NULL 类似。

```
// 2. 查找现有映射
list_for_each_entry(entry, head, list) {
    if (dir == DIR_OUT) {
        // 出站匹配: 源IP/端口 (内网) + 对端IP/端口
        if (entry->internal_ip == saddr && entry->internal_port == sport &&
            entry->remote_ip == remote_ip && entry->remote_port == remote_port) {
            mapping = entry;
            break;
        }
    } else { // DIR_IN
        // 入站匹配: 目的IP/端口 (公网) + 对端IP/端口
        if (entry->external_ip == daddr && entry->external_port == dport &&
            entry->remote_ip == remote_ip && entry->remote_port == remote_port) {
            mapping = entry;
            break;
        }
    }
}
```

若数据包方向为 DIR_OUT,则同样先寻找到是否有匹配的映射项。若 mapping 不为 NULL,则同样更新映射项的信息,包括内部序列号、ACK、FIN 标志等。如果未找到匹配的映射项,说明是新的连接,需要分配一个外部端口,并创建新的映射项。遍历 NAT_PORT_MIN 到 NAT_PORT_MAX 的端口号,找到第一个未被分配的端口。如果找到可用端口,将其标记为已分配,并将新的映射项的信息设置为新分配的端口和出站方向数据包的源 IP 和源端口。而后也更新映射表的信息。

```
// 3. 处理映射未找到的情况 (Miss)
if (mapping == NULL) {
    if (dir == DIR_OUT) {
        // SNAT: 分配新端口并建立映射
        int i;
        u16 assigned_port = 0;
        for (i = NAT_PORT_MIN; i <= NAT_PORT_MAX; i++) {
            if (nat.assigned_ports[i] == 0) {
```

```

        nat.assigned_ports[i] = 1;
        assigned_port = i;
        break;
    }
}
// ... 创建新映射 ...
} else { // DIR_IN
    // DNAT: 查找匹配的DNAT规则
    // ... 遍历规则并创建映射 ...
}
}
}

```

最终,更新连接状态后,执行地址转换:对于出站数据包,修改源 IP 和源端口;对于入站数据包,修改目的 IP 和目的端口。然后重算 IP 校验和和 TCP 校验和,最后发送数据包。

```

// 4. 更新连接状态 (SEQ/ACK/FIN)
// ... 更新代码 ...

// 5. 执行地址转换
if (dir == DIR_OUT) {
    ip->saddr = htonl(mapping->external_ip);
    tcp->sport = htons(mapping->external_port);
} else {
    ip->daddr = htonl(mapping->internal_ip);
    tcp->dport = htons(mapping->internal_port);
}

ip->checksum = ip_checksum(ip);
tcp->checksum = tcp_checksum(ip, tcp);

ip_send_packet(packet, len);

```

2.3 清除超时的 NAT 映射表项

nat_timeout 函数是一个后台线程函数,用于定期清除超时的 NAT 映射表项,以避免内存泄漏和端口耗尽。该函数在一个无限循环中运行,每秒执行一次清理操作。首先,函数锁定互斥锁以确保线程安全,然后获取当前时间。

```

void *nat_timeout()
{
    while (1) {
        pthread_mutex_lock(&nat.lock);
        time_t now = time(NULL);
        // ... 遍历和清理 ...
        pthread_mutex_unlock(&nat.lock);
        sleep(1);
    }
    return NULL;
}

```

函数遍历所有哈希桶(从 0 到 HASH_8BITS-1),对于每个哈希桶的链表,使用 list_for_each_entry_safe 安全地遍历映射项。对于每个映射项,检查是否满足清理条件:连接是否已完成(通过 is_flow_finished(&mapping->conn) 判断)或自上次更新以来是否超过 TCP_ESTABLISHED_TIMEOUT 秒。

```
for (int i = 0; i < HASH_8BITS; i++) {
    struct list_head *head = &nat.nat_mapping_list[i];
    struct nat_mapping *mapping, *q;
    list_for_each_entry_safe(mapping, q, head, list) {
        if (is_flow_finished(&mapping->conn) || \
            (now - mapping->update_time >= TCP_ESTABLISHED_TIMEOUT)) {
            // ... 清理操作 ...
        }
    }
}
```

如果满足条件,则释放分配的端口(如果外部端口在 NAT_PORT_MIN 到 NAT_PORT_MAX 范围内,将其标记为未分配),然后从链表中移除映射项并释放内存。最后,解锁互斥锁并睡眠 1 秒,等待下一次清理。

```
if (mapping->external_port >= NAT_PORT_MIN && mapping->external_port <= NAT_PORT_MAX) {
    nat.assigned_ports[mapping->external_port] = 0;
}
list_delete_entry(&mapping->list);
free(mapping);
```

2.4 解析配置文件

parse_config 函数用于解析 NAT 配置文件,包括内部接口、外部接口以及 DNAT 规则。该函数打开指定的配置文件,逐行读取并解析内容。首先,函数使用 fopen 以二进制模式打开文件,然后进入循环读取每一行,直到文件结束或出错。

```
int parse_config(const char *filename)
{
    char line[256];
    FILE *fp = fopen(filename, "rb");
    char type[128], name[128], exter[64], inter[64];
    // ... 解析接口 ...
    // ... 解析规则 ...
    return 0;
}
```

在第一个循环中,函数读取配置文件的接口部分,使用 sscanf 解析每一行的类型和名称。如果类型为"internal-iface",则通过 if_name_to_iface 获取内部接口并赋值给 nat.internal_iface;如果类型为"external-iface",则获取外部接口并赋值给 nat.external_iface。如果解析失败,则输出错误信息。

```
while (!feof(fp) && !ferror(fp)) {
    strcpy(line, "\n");
    fgets(line, sizeof(line), fp);
    if (line[0] == '\n') break;
    sscanf(line, "%s %s", type, name);
```

```

type[14] = '\0';
if (strcmp(type, "internal-iface") == 0) {
    printf("Internal-iface: %s .\n", name);
    nat.internal_iface = if_name_to_iface(name);
} else if (strcmp(type, "external-iface") == 0) {
    printf("External-iface: %s .\n", name);
    nat.external_iface = if_name_to_iface(name);
} else printf("config iface failed : %s .\n", type);
}

```

在第二个循环中，函数读取 DNAT 规则部分。对于每一行，如果类型为”dnat-rules”，则分配新的 `dnat_rule` 结构体并添加到 `nat.rules` 链表中。然后，解析外部 IP 和端口：使用 `sscanf` 提取 IP 字符串和端口号，再将 IP 字符串转换为 `u32` 格式（通过位移运算组合四个字节）。同样解析内部 IP 和端口，并输出调试信息。如果解析失败，则输出错误信息。

```

u32 ip4, ip3, ip2, ip1, ip;
u16 port;
while (!feof(fp) && !ferror(fp)) {
    strcpy(line, "\n");
    fgets(line, sizeof(line), fp);
    if (line[0] == '\n') break;
    sscanf(line, "%s %s %s %s", type, exter, name, inter);
    type[10] = '\0';
    if (strcmp(type, "dnat-rules") == 0) {
        // ... 分配和解析规则 ...
        sscanf(exter, "%[^:]:%hu", name, &port);
        sscanf(name, "%u.%u.%u.%u", &ip4, &ip3, &ip2, &ip1);
        ip = (ip4 << 24) | (ip3 << 16) | (ip2 << 8) | (ip1);
        rule->external_ip = ip;
        rule->external_port = port;
        // ... 类似解析内部 ...
    } else printf("config rules failed : %s .\n", type);
}

```

2.5 释放 NAT 相关资源

`nat_exit` 函数用于在程序退出时释放所有分配的 NAT 相关资源，包括映射表项和后台线程。该函数首先锁定互斥锁以确保线程安全，然后遍历所有哈希桶（从 0 到 `HASH_8BITS-1`），对于每个哈希桶的链表，使用 `list_for_each_entry_safe` 安全地遍历并释放映射项。

```

void nat_exit()
{
    pthread_mutex_lock(&nat.lock);
    for (int i = 0; i < HASH_8BITS; i++) {
        struct nat_mapping *entry, *q;
        list_for_each_entry_safe(entry, q, &nat.nat_mapping_list[i], list) {
            list_delete_entry(&entry->list);
            free(entry);
        }
    }
}

```

```

}

pthread_kill(nat.thread, SIGTERM);
pthread_mutex_unlock(&nat.lock);
}

```

对于每个映射项，函数从链表中移除并释放内存。然后，函数向后台线程(nat.thread)发送 SIGTERM 信号以终止线程。最后，解锁互斥锁。这种设计确保了资源完全释放，避免内存泄漏和线程残留。

三、实验结果

3.1 实验一:SNAT 实验

在 h1 和 h2 上分别运行 wget 命令访问 h3 的 HTTP 服务,成功获取页面,说明 SNAT 功能正常工作。

```

11-nat > code > log > ≡ h1_snat.log
1  --2025-12-10 09:44:55-- http://159.226.39.123:8000/
2  Connecting to 159.226.39.123:8000... connected.
3  HTTP request sent, awaiting response... 200 OK
4  Length: 212 [text/html]
5  Saving to: 'index.html.2.1'
6
7      OK                                     100% 17.1M=0s
8
9  2025-12-10 09:44:56 (17.1 MB/s) - 'index.html.2.1' saved [212/212]
10
11

11-nat > code > log > ≡ h2_snat.log
1  --2025-12-10 09:44:55-- http://159.226.39.123:8000/
2  Connecting to 159.226.39.123:8000... connected.
3  HTTP request sent, awaiting response... 200 OK
4  Length: 212 [text/html]
5  Saving to: 'index.html.2'
6
7      OK                                     100% 34.2M=0s
8
9  2025-12-10 09:44:56 (34.2 MB/s) - 'index.html.2' saved [212/212]
10
11

11-nat > code > log > ≡ h3_snat.log
1  159.226.39.43 - - [10/Dec/2025 09:44:56] "GET / HTTP/1.1" 200 -
2  159.226.39.43 - - [10/Dec/2025 09:44:56] "GET / HTTP/1.1" 200 -
3

11-nat > code > log > ≡ n1_snat.log
1  DEBUG: find the following interfaces: n1-eth1 n1-eth0.
2

```

图 1: SNAT 实验结果

3.2 实验二:DNAT 实验

在 h3 上分别运行 wget 命令访问 h1 和 h2 的 HTTP 服务,成功获取页面,说明 DNAT 功能正常工作。

```
11-nat > code > log > ≡ h3_dnat_8000.log
1  --2025-12-10 09:44:23-- http://159.226.39.43:8000/
2  Connecting to 159.226.39.43:8000... connected.
3  HTTP request sent, awaiting response... 200 OK
4  Length: 208 [text/html]
5  Saving to: 'index.html.1.1'
6
7      OK                                     100% 101M=0s
8
9  2025-12-10 09:44:24 (101 MB/s) - 'index.html.1.1' saved [208/208]
10
11

11-nat > code > log > ≡ h3_dnat_8001.log
1  --2025-12-10 09:44:23-- http://159.226.39.43:8001/
2  Connecting to 159.226.39.43:8001... connected.
3  HTTP request sent, awaiting response... 200 OK
4  Length: 208 [text/html]
5  Saving to: 'index.html.1'
6
7      OK                                     100% 87.1M=0s
8
9  2025-12-10 09:44:24 (87.1 MB/s) - 'index.html.1' saved [208/208]
10
11

11-nat > code > log > ≡ h1_dnat.log
1  159.226.39.123 - - [10/Dec/2025 09:44:24] "GET / HTTP/1.1" 200 -
2

11-nat > code > log > ≡ h2_dnat.log
1  159.226.39.123 - - [10/Dec/2025 09:44:24] "GET / HTTP/1.1" 200 -
2

11-nat > code > log > ≡ n1_dnat.log
1  DEBUG: find the following interfaces: n1-eth1 n1-eth0.
2
```

图 2: DNAT 实验结果

3.3 实验三:自建双 NAT 拓扑实验

网络拓扑构建如下:

```
...
class myTopo(Topo):
    def build(self):
        h1 = self.addHost('h1')
        h2 = self.addHost('h2')
        n1 = self.addHost('n1')
        n2 = self.addHost('n2')

        # h1-eth0 <-> n1-eth0
        self.addLink(h1, n1)
        # n1-eth1 <-> n2-eth0
        self.addLink(n1, n2)
```



```

        # n2-eth1 <-> h2-eth0
        self.addLink(n2, h2)

if __name__ == '__main__':
    check_scripts()

    topo = myTopo()
    net = Mininet(topo = topo, switch = OVSBridge, controller = None)
    h1, h2, n1, n2 = net.get('h1', 'h2', 'n1', 'n2')

    # ===== IP & 路由配置 =====

    # 内网 A: h1 <-> n1
    h1.cmd('ifconfig h1-eth0 10.21.0.1/16')
    h1.cmd('route add default gw 10.21.0.254')
    n1.cmd('ifconfig n1-eth0 10.21.0.254/16')

    # 中间网: n1 <-> n2
    n1.cmd('ifconfig n1-eth1 159.226.39.43/24')
    n2.cmd('ifconfig n2-eth0 159.226.39.44/24')

    # 内网 B: n2 <-> h2
    h2.cmd('ifconfig h2-eth0 10.22.0.1/16')
    h2.cmd('route add default gw 10.22.0.254')
    n2.cmd('ifconfig n2-eth1 10.22.0.254/16')

    # ===== 关闭内核协议栈功能, 交给用户态 NAT =====
    for n in (n1, n2):
        n.cmd('./scripts/disable_arp.sh')
        n.cmd('./scripts/disable_icmp.sh')
        n.cmd('./scripts/disable_ip_forward.sh')
        n.cmd('./scripts/disable_ipv6.sh')

    for node in (h1, h2, n1, n2):
        node.cmd('./scripts/disable_offloading.sh')
        node.cmd('./scripts/disable_ipv6.sh')

    net.start()

    # ===== 启动 NAT / HTTP / 客户端请求 =====
    # n1: SNAT
    n1.cmd('./nat n1.conf > ./log/n1_snat_3.log 2>&1 &')
    # n2: DNAT
    n2.cmd('./nat n2.conf > ./log/n2_dnat_3.log 2>&1 &')
    # h2: HTTP server
    h2.cmd('python3 ./http_server.py > ./log/h2_http_3.log 2>&1 &')
    # h1: 访问 n2 的“公网”地址
    h1.cmd('wget http://159.226.39.44:8000 > ./log/h1_wget_3.log 2>&1 &')

    CLI(net)

```

```
net.stop()
```

配置文件如下:

```
# n1.conf (SNAT)
internal-iface: n1-eth0
external-iface: n1-eth1

# n2.conf (DNAT)
internal-iface: n2-eth1
external-iface: n2-eth0

dnat-rules: 159.226.39.44:8000 -> 10.22.0.1:8000
```

运行结果如下, h1 成功访问 h2 的 HTTP 服务, 说明双 NAT 功能正常工作。

```
11-nat > code > log > ≡ h1_wget_3.log
1  --2025-12-11 01:58:42-- http://159.226.39.44:8000/
2  Connecting to 159.226.39.44:8000... connected.
3  HTTP request sent, awaiting response... 200 OK
4  Length: 207 [text/html]
5  Saving to: 'index.html.4'
6
7      OK                                     100% 513K=0s
8
9  2025-12-11 01:58:43 (513 KB/s) - 'index.html.4' saved [207/207]
10
11

11-nat > code > log > ≡ h2_http_3.log
1  159.226.39.43 - - [11/Dec/2025 01:58:43] "GET / HTTP/1.1" 200 -
2

11-nat > code > log > ≡ n1_snat_3.log
1  DEBUG: find the following interfaces: n1-eth0 n1-eth1.
2

11-nat > code > log > ≡ n2_dnat_3.log
1  DEBUG: find the following interfaces: n2-eth0 n2-eth1.
2
```

图 3: 双 NAT 实验结果